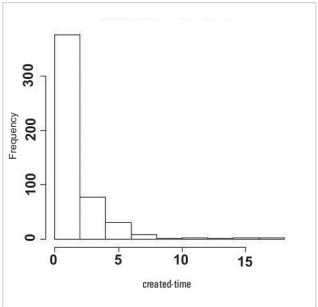

Figure 3: Histogram of ordered created time tag

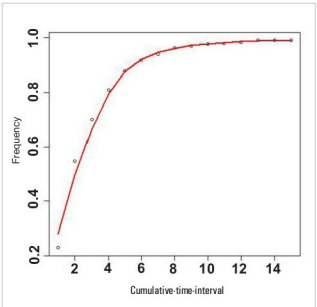

Figure 4: Cumulative frequency distribution

```
 $ screenName       : chr "Lezzardman"
 $ retweetCount     : num 0
 $ isRetweet         : logi FALSE
 $ retweeted         : logi FALSE
 $ longitude          : chr NA
 $ latitude             : chr NA
 $ location            : chr "Bay Area, CA, #CLGWORLDWIDE
<ed><U+00AB><U+00BD><ed><U+00B2><U+00AF>"
 $ language          : chr "en"
 $ profileImageURL: chr "http://pbs.twimg.com/profile_
images/444325116407603200/XmZ92Dv8_normal.jpeg"
>
```

The fifth column field is 'created'; we shall try to explore the different statistical characteristics features of this field.

```
>attach(loveDF)         # attach the frame for further
processing.
>head(loveDF['created'],2)  # first 2 record set items for
demo.
created
1 2017-10-04 06:11:03
2 2017-10-04 06:10:55
```

Twitter follows the Coordinated Universal Time tag as the time-stamp to record the tweet's time of creation. This helps to maintain a normalised time frame for all records, and it becomes easy to draw a frequency histogram of the 'created' time tag.

```
>hist(created,breaks=15,freq=TRUE,main="Histogram of
created time tag")
```

If we want to study the pattern of how the word 'love' appears in the data set, we can take the differences of consecutive time elements of the vector 'created'. R function *diff()* can do this. It returns iterative lagged differences of the elements of an integer vector. In this case, we need lag and iteration variables as one. To have a time series from the 'created' vector, it first needs to be converted to an integer; here, we have done it before creating the series, as follows:

```
>detach(loveDF)
>sortloveDF<-loveDF[order(as.integer(created)),]
>attach(sortloveDF)
>hist(as.integer(abs(diff(created))))
```

This distribution shows that the majority of tweets in this group come within the first few seconds and a much smaller number of tweets arrive in subsequent time intervals. From the distribution, it's apparent that the arrival time distribution follows a Poisson Distribution pattern, and it is now possible to model the number of times an event occurs in a given time interval.

Let's check the cumulative distribution pattern, and the number of tweets arriving within a time interval. For this we have to write a short R function to get the cumulative values within each interval. Here is the demo script and the graph plot:

```
countarrival<- function(created)
{
i=1
s <- seq(1,15,1)
for(t in seq(1,15,1))
```